



移动智能终端生态联盟——大屏适配白皮书

Jetpack Compose 适配指导

编制单位：移动智能终端生态联盟·标准工作组

2025 年 5 月

目录

1. 响应式布局.....	4
1.1 均分能力.....	4
1.2 占比能力.....	5
1.3 缩放能力.....	6
1.4 相对拉伸.....	6
1.5 内容延伸.....	10
2. 自适应布局.....	13
2.1 断点.....	14
2.2 栅格.....	17
3. 典型布局场景.....	20
3.1 构建自适应导航.....	20
3.2 构建列表详情布局.....	24
3.3 构建辅助窗格布局.....	28
3.4 缩进布局.....	31
3.5 增列布局:	31
3.6 挪移布局.....	32
3.7 深度定制布局.....	33
4. 自适应注意事项.....	35
4.1 去除宽高比的限制.....	35
4.2 移除固定方向的限制.....	35
4.3 尺寸可调整.....	38
4.4 感知窗口大小, 而非屏幕.....	39
4.5 动态感知大小, 而非静态.....	42

布局可以分为自适应布局和响应式布局，二者的介绍如下表所示。

名称	简介
响应式布局	响应式布局（Responsive Layout），是指让页面元素根据窗口尺寸的变化实时调整位置，像水一样动态适配不同大小的窗口，常见的动态响应式布局包括：均分能力，占比能力，缩放能力，相对拉伸，内容延伸
自适应布局	自适应布局（Adaptive Layout），是通过系统提供的自适应布局能力，为不同类型的窗口创建多种布局，并根据窗口尺寸类型选择对应的布局，当窗口尺寸跨越某个边界值时，自动改变页面的布局，常见的自适应布局包括缩进布局，增列布局，挪移布局以及深度定制布局等

说明：

响应式布局多用于解决页面各区域内的布局差异，自适应布局多用于解决页面各区域间的布局差异。

自适应布局和响应式布局常常需要借助容器类组件实现，或与容器类组件搭配使用。

响应式布局常常需要借助 Row 组件、Column 组件 实现。

容器组件	组件说明	相对拉伸	缩放能力	内容延伸	均分能力	占比能力
Row	沿水平方向布局子组件的容器	通过添加 Arrangement 属性实现	配置组件 aspectRatio 属性	使用 LazyRow、LazyHorizontalGrid	使用 Arrangement 空间分配	使用 Modifier 权重分配
Column	沿垂直方向布局子组件的容器	通过添加 Arrangement 属性实现	配置组件 aspectRatio 属性	使用 LazyColumn、LazyVerticalGrid	使用 Arrangement 空间分配	使用 Modifier 权重分配

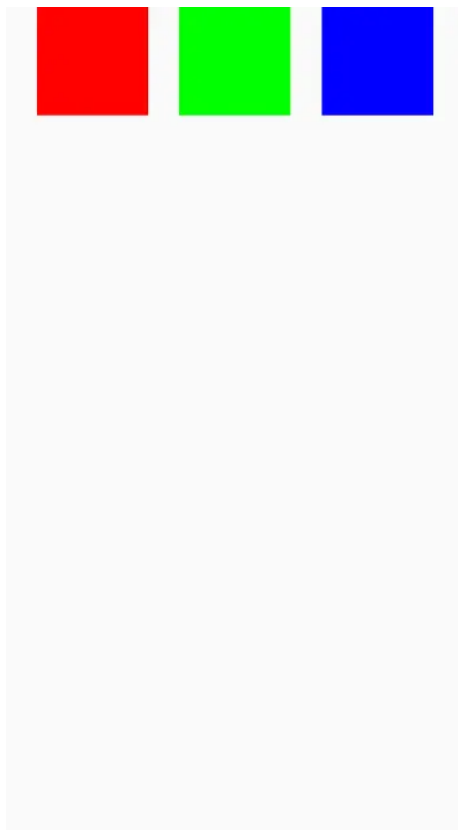
1. 响应式布局

1.1 均分能力

均分能力是指容器组件尺寸发生变化时，增加或减小的空间均匀分配给容器组件内所有空白区域。它常用于内容数量固定、均分显示的场景，比如工具栏、底部菜单栏等。

可通过设置 Row/Column 的 `horizontalArrangement/verticalArrangement` 参数为 `Arrangement.SpaceEvenly`，该方案会将容器宽度均分为 $n+1$ 等份（ n 为子元素数量），实现元素间距相等的效果。

示例：



```
Row(LayoutSize.Fill, arrangement = Arrangement.SpaceEvenly) {  
    Box(LayoutSize(100.dp), backgroundColor = Color.Red)  
    Box(LayoutSize(100.dp), backgroundColor = Color.Green)  
    Box(LayoutSize(100.dp), backgroundColor = Color.Blue)}
```

Java

1.2 占比能力

占比能力是指子组件的宽高按照预设的比例，随父容器组件发生变化。

可以通过 `Modifier.weight` 实现比例分配，您可以使用仅在 `RowScope` 和 `ColumnScope` 中使用的 `weight` 修饰符，将可组合项的尺寸设置为可在其父项内灵活调整。

示例：

以包含两个 `Box` 可组合项的 `Row` 为例。第一个框的 `weight` 是第二个框的两倍，因此其宽度也相差两倍。



**Alfred
Sisley**
3 minutes ago

```
@Composable
fun ArtistCard(/*...*/) {
    Row(
        modifier = Modifier.fillMaxWidth()
    ) {
        Image(
            /*...*/
            modifier = Modifier.weight(2f)
        )
        Column(
            modifier = Modifier.weight(1f)
        ) {
            /*...*/
        }
    }
}
```

Java

1.3 缩放能力

指页面内元素保持固定比例，通过对比参照物确定尺寸，屏幕变宽时，元素会随之等比变大。缩放能力通过使用百分比布局配合**固定宽高比**（`Modifier.aspectRatio()` 属性）实现当容器尺寸发生变化时，内容自适应调整。

示例：

将图片转换为自定义宽高比

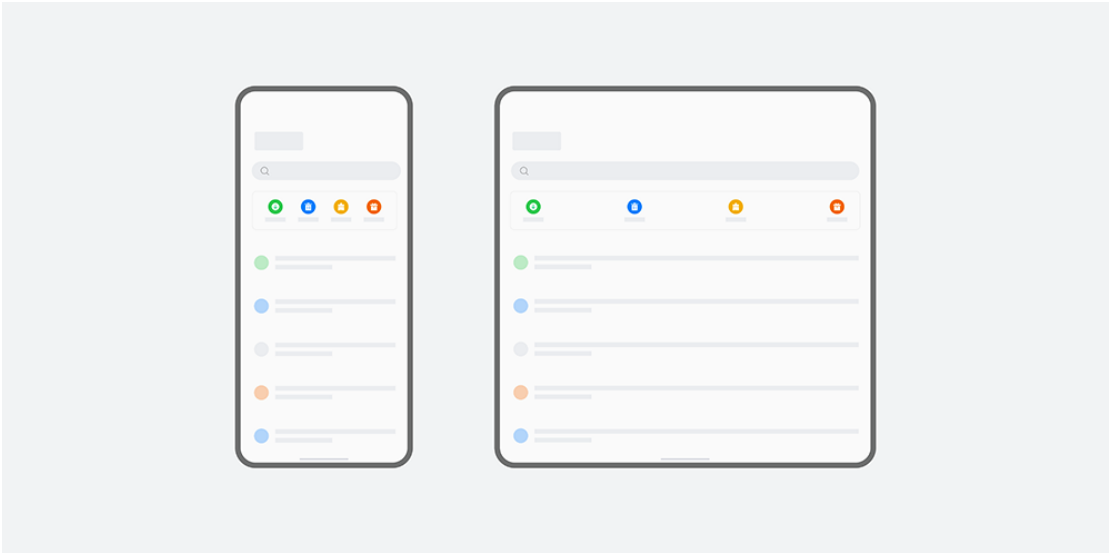


```
Image(  
    painter = painterResource(id = R.drawable.dog),  
    contentDescription = stringResource(id = R.string.dog_content_description),  
    modifier = Modifier.aspectRatio(16f / 9f))
```

Java

1.4 相对拉伸

指页面内元素高度固定，宽度通过对比参照物确定，页面变宽时，元素的宽度或多个元素之间的间距随之变宽



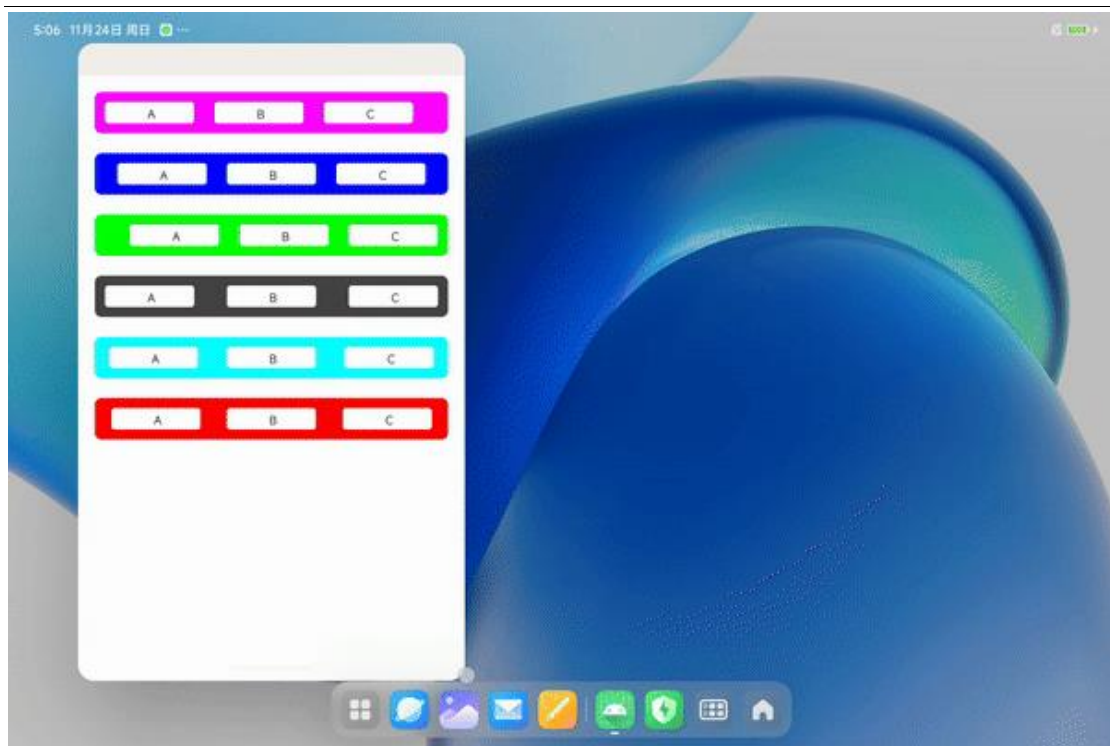
适配方法：

通过 Row（水平排列）和 Column（竖向排列）的 Arrangement 属性实现

对齐方式	排列方式
Space Between	等宽/等高排列
Space Around	等间距排列
Space Evenly	每个 item 等 padding 排列
Start	头部对齐排列
Center	居中排列
End	尾部对齐排列

示例：

通过 Row 不同的 Arrangement 属性，实现不同的排列效果



代码实现：

```
// 设置显示元素@Composable
fun ShowElement (
    text: String,
    modifier: Modifier = Modifier) {
    Surface(
        color = Color.White,
        shape = RoundedCornerShape(4.dp),
        modifier = modifier.widthIn(130.dp).heightIn(50.dp).padding(12.dp),
    ) {
        Row {
            Spacer(Modifier.padding(horizontal = 25.dp))
            Text(text, modifier.paddingFromBaseline(20.dp))
        }
    }
}

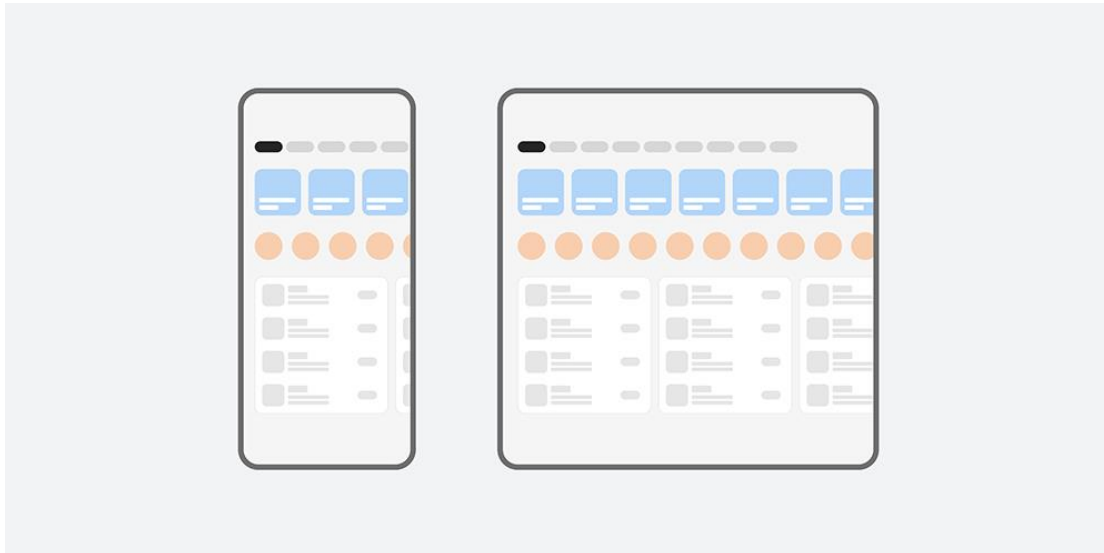
// 设置元素的排列方式@Composable
fun ShowElementRow (
    color: Color,
    horizontalArrangement: Arrangement.Horizontal,
```

```
modifier: Modifier = Modifier) {  
    Surface(  
        shape = RoundedCornerShape(8.dp),  
        modifier = Modifier  
            .fillMaxWidth()  
            .padding(12.dp)  
    ) {  
        Row(  
            horizontalArrangement = horizontalArrangement,  
            modifier = Modifier  
                .background(color)  
                .fillMaxWidth(),  
        ) {  
            ShowElement("A", modifier)  
            ShowElement("B", modifier)  
            ShowElement("C", modifier)  
        }  
    }  
}  
}}@Preview@Composable private fun ShowElementRowPreview (  
    modifier: Modifier = Modifier) {  
    Column(  
        modifier = modifier.padding(8.dp)  
    ) {  
        // 设置六种不同的排列方式  
        ShowElementRow(Color.Magenta, Arrangement.Start, modifier)  
        ShowElementRow(Color.Blue, Arrangement.Center, modifier)  
        ShowElementRow(Color.Green, Arrangement.End, modifier)  
        ShowElementRow(Color.DarkGray, Arrangement.SpaceBetween, modifier)  
        ShowElementRow(Color.Cyan, Arrangement.SpaceAround, modifier)  
        ShowElementRow(Color.Red, Arrangement.SpaceEvenly, modifier)  
    }  
}}
```

Java

1.5 内容延伸

指当屏幕变宽时，像展开画卷一样，露出数量更多的元素



适配方法：

在 Compose 中，使用 `LazyRow` 和 `LazyColumn` 可以使行和列中的元素进行滑动显示，而在窗口宽度逐渐变宽时，会逐渐显示更多的元素，实现元素的延伸显示效果。除了行和列的延伸显示，`LazyHorizontalGrid` 和 `LazyVerticalGrid` 还可以实现水平网格排列和竖向网格排列下的元素延伸效果，并且以上方法只会渲染屏幕上显示的元素（而不是同时渲染所有元素），可以避免预加载影响性能。

使用方法如下：

`LazyRow`

`state`：用于控制或观察列表状态的状态对象

`contentPadding`：整个内容周围的填充 `HorizontalArrangement`。

`reverseLayout`：反转滚动和布局的方向，当为 `true` 时，项目将以相反的顺序排列

`horizontalArrangement`：布局子项的水平排列方式（默认 `Arrangement.Start`）

`verticalAlignment`：垂直对齐方式（默认为 `Alignment.Top`）

`LazyColumn`

`state`：用于控制或观察列表状态的状态对象

`contentPadding` : 整个内容周围的填充 `HorizontalArrangement`。

`reverseLayout` : 反转滚动和布局的方向, 当为 `true` 时, 项目将以相反的顺序排列

`verticalArrangement` : 布局子项的水平排列方式 (默认 `Arrangement.Top`)

`horizontalAlignment` : 垂直对齐方式 (默认为 `Alignment.Start`)

LazyHorizontalGrid

`rows` : 单元格中的行数

`state` : 用于控制或观察列表状态的状态对象

`contentPadding` : 整个内容周围的填充

`reverseLayout` : 反转滚动和布局的方向, 当为 `true` 时, 项目将以相反的顺序排列

`verticalArrangement` : 布局子项的垂直排列方式 (默认 `Arrangement.Top`)

`horizontalArrangement` : 布局子项的水平排列方式 (默认 `Arrangement.Start`)

LazyVerticalGrid

`columns` : 单元格中的列数

`state` : 用于控制或观察列表状态的状态对象

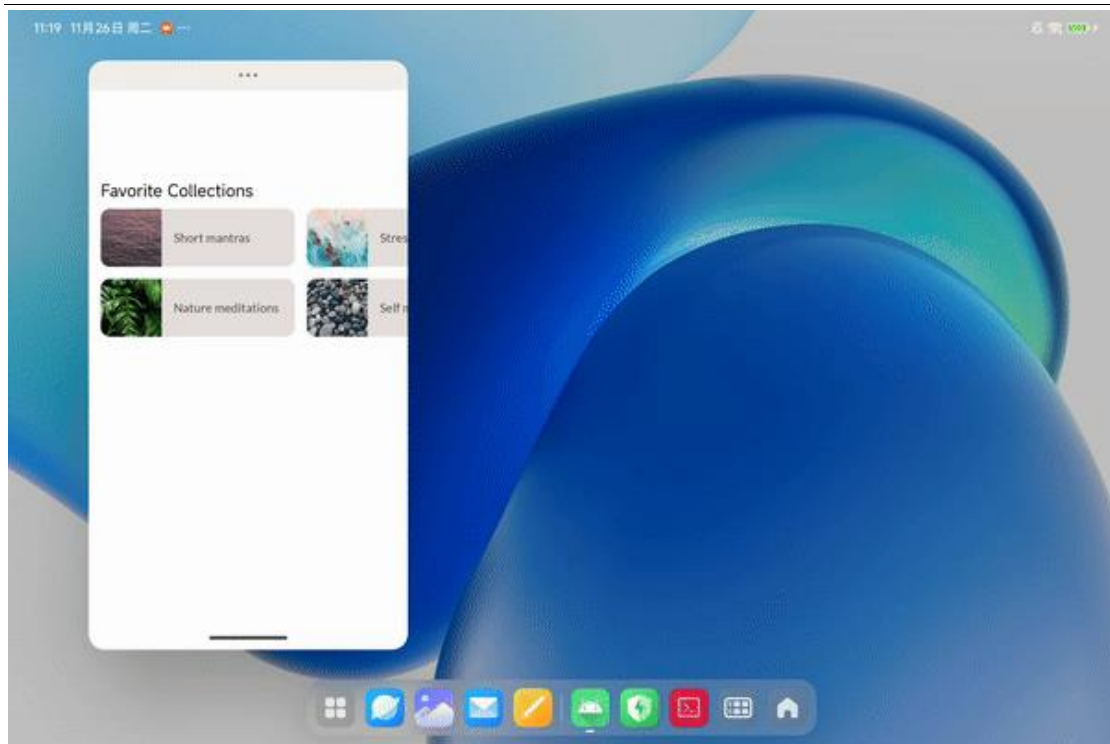
`contentPadding` : 整个内容周围的填充

`reverseLayout` : 反转滚动和布局的方向, 当为 `true` 时, 项目将以相反的顺序排列

`verticalArrangement` : 布局子项的垂直排列方式 (默认 `Arrangement.Top`)

`horizontalArrangement` : 布局子项的水平排列方式 (默认 `Arrangement.Start`)

LazyHorizontalGrid 实现:



代码实现：

```
@Composable
fun FavoriteCollectionsGrid(
    modifier: Modifier = Modifier) {
    Column(modifier) {
        Spacer(modifier = modifier.padding(vertical = 50.dp))
        Text(
            text = stringResource(R.string.favorite_collections),
            style = MaterialTheme.typography.titleLarge,
            modifier = Modifier
                .paddingFromBaseline(top = 40.dp, bottom = 16.dp)
                .padding(horizontal = 16.dp)
        )
        LazyHorizontalGrid(
            rows = GridCells.Fixed(2),
            contentPadding = PaddingValues(horizontal = 16.dp),
            horizontalArrangement = Arrangement.spacedBy(16.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp),
            modifier = modifier.height(168.dp)
        )
    }
}
```

```
    ) {  
        items(favoriteCollectionsData) { item ->  
            FavoriteCollectionCard(item.drawable, item.text,  
                Modifier.height(80.dp))  
        }  
    }  
}  
}  
  
} // 预览效果@Preview(showBackground = true, backgroundColor =  
0xFFFF5F0EE) @Composable  
  
fun FavoriteCollectionsGridPreview() {  
    FavoriteCollectionsGrid() // 加载图片 private val favoriteCollectionsData =  
listOf(  
    R.drawable.fc1_short_mantras to R.string.fc1_short_mantras,  
    R.drawable.fc2_nature_meditations to R.string.fc2_nature_meditations,  
    R.drawable.fc3_stress_and_anxiety to R.string.fc3_stress_and_anxiety,  
    R.drawable.fc4_self_message to R.string.fc4_self_message,  
    R.drawable.fc5_overwhelmed to R.string.fc5_overwhelmed,  
    R.drawable.fc6_nightly_wind_down to R.string.fc6_nightly_wind_down,  
    R.drawable.fc7_nature_meditations to R.string.fc7_nature_meditations,  
    R.drawable.fc8_short_mantras to R.string.fc8_short_mantras).map  
{ DrawableStringPair(it.first, it.second) }
```

Java

2. 自适应布局

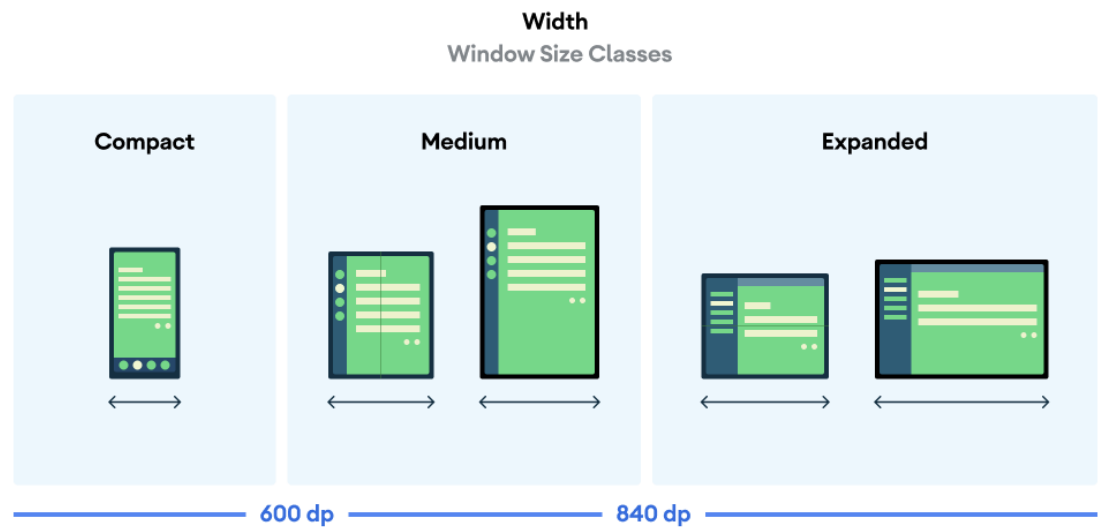
自适应布局（Adaptive Layout），是通过系统提供的自适应布局能力，为不同类型的窗口创建多种布局，并根据窗口尺寸类型选择对应的布局，当窗口尺寸跨越某个边界值时，自动改变页面的布局，常见的自适应布局包括缩进布局，增列布局，挪移布局以及深度定制布局等

自适应布局能力	简介
断点	将窗口宽度划分为不同的范围（即断点），监听窗口尺寸变

	化，当断点改变时同步调整页面布局。
栅格	栅格组件将其所在的区域划分为有规律的多列，通过调整不同断点下的栅格组件的参数以及其子组件占据的列数等，实现不同的布局效果。

2.1 断点

窗口尺寸类别是一个主观断点，即布局需要更改以匹配可用空间、设备惯例和人体工程学的窗口尺寸。设计时无需考虑越来越多的显示状态，而是关注窗口类别大小，以确保布局能够在各种设备上运行，断点分为：compact, medium, expanded，通常情况下我们只需要考虑前三种，可以通过使用 `currentWindowAdaptiveInfo().windowSizeClass` 获取对应的属性进行判断。



小类别图示。

基于宽度的窗口大



基于高度的窗口大小类别图

示。

如图所示，这些断点可让您继续从设备和配置的角度考虑布局。每个大小类别划分点代表了典型设备场景的大多数情况，当您考虑基于划分点的布局设计时，下表是一个有用的参考框架。

大小类别	划分点	设备表示
较小的宽度	宽度 < 600dp	99.96% 的手机处于竖屏模式
中等宽度	600dp ≤ 宽度 < 840dp	93.73% 的平板电脑处于竖屏模式，大多数展开的大型内部显示屏处于竖屏模式
较大宽度	宽度 ≥ 840dp	97.22% 的平板电脑处于横屏模式，大多数展开的大型内部显示屏处于横屏模式
较小的高度	高度 < 480dp	99.78% 的手机处于横屏模式

中等高度	$480\text{dp} \leq \text{高度} < 900\text{dp}$	96.56% 的平板电脑处于横屏模式，97.59% 的手机处于竖屏模式
展开高度	$\text{高度} \geq 900\text{dp}$	94.25% 的平板电脑处于竖屏模式

注意：大多数应用在构建自适应界面时可以只考虑宽度窗口大小类别。

可以使用 `WindowSizeClass#compute()` 计算当前的 `WindowSizeClass`。以下示例显示了如何计算窗口大小类别，并在窗口大小类别变更：

```
class MainActivity : Activity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // ...
        // Replace with a known container that you can safely add a
        // view to where the view won't affect the layout and the view
        // won't be replaced.
        val container: ViewGroup = binding.container
        // Add a utility view to the container to hook into
        // View.onConfigurationChanged(). This is required for all
        // activities, even those that don't handle configuration
        // changes. You can't use Activity.onConfigurationChanged(),
        // since there are situations where that won't be called when
        // the configuration changes. View.onConfigurationChanged() is
        // called in those scenarios.
        container.addView(object : View(this) {
            override fun onConfigurationChanged(newConfig: Configuration?) {
                super.onConfigurationChanged(newConfig)
                computeWindowSizeClasses()
            }
        })
        computeWindowSizeClasses()
    }
}
```

```
    }  
  
    private fun computeWindowSizeClasses() {  
        val metrics =  
WindowMetricsCalculator.getOrCreate().computeCurrentWindowMetrics(this)  
  
        val width = metrics.bounds.width()  
        val height = metrics.bounds.height()  
        val density = resources.displayMetrics.density  
        val windowSizeClass = WindowSizeClass.compute(width/density,  
height/density)  
  
        // COMPACT, MEDIUM, or EXPANDED  
        val widthWindowSizeClass = windowSizeClass.windowWidthSizeClass  
  
        // COMPACT, MEDIUM, or EXPANDED  
        val heightWindowSizeClass = windowSizeClass.windowHeightSizeClass  
  
        // Use widthWindowSizeClass and heightWindowSizeClass.  
    }  
}
```

Java

2.2 栅格

栅格是对元素横向排布的一种辅助，以规则的列数、边距和间距，来指导元素的分布。使用栅格进行布局，可以使元素排列更有秩序感，有助于让应用在不同的设备上呈现相似的布局，或呈现更明确的开发规则。通常对于栅格布局，我们可以考虑使用 `LazyHorizontalGrid` 和 `LazyVerticalGrid` 来实现，通过对于尺寸的感知，来动态的生成行数或列数实现对应的效果



固定列数布局

使用 `GridCells.Fixed(n)` 可创建固定列数的网格，例如实现三列布局：

```
LazyVerticalGrid(columns = GridCells.Fixed(3)) { /* items */ }
```

自适应列数布局

通过 `GridCells.Adaptive(minSize)` 实现根据屏幕宽度动态调整列数，例如每列最小宽度为 80dp：

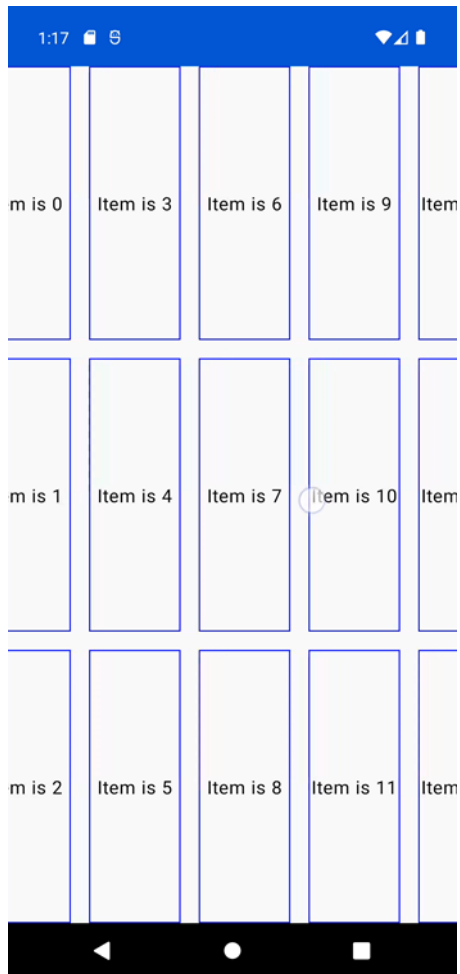
```
LazyVerticalGrid(columns = GridCells.Adaptive(80.dp)) { /* items */ }
```

行列间距控制

支持通过 `horizontalArrangement` 和 `verticalArrangement` 参数设置行列间距，例如：

```
LazyVerticalGrid(columns = ..., horizontalArrangement =  
Arrangement.spacedBy(16.dp))
```

示例：



解释

@Composable

```
fun ScrollingGrid() {  
    val itemList = (0..15).toList()  
  
    val itemModifier = Modifier  
        .border(1.dp, Color.Blue)  
        .width(80.dp)  
        .wrapContentSize()  
  
    LazyHorizontalGrid(  
        rows = GridCells.Fixed(3),  
        horizontalArrangement = Arrangement.spacedBy(16.dp),  
    )  
}
```

```
verticalArrangement = Arrangement.spacedBy(16.dp)

) {

    items(itemsList) {

        Text("Item is $it", itemModifier)

    }

    item {

        Text("Single item", itemModifier)

    }

}}
```

Java

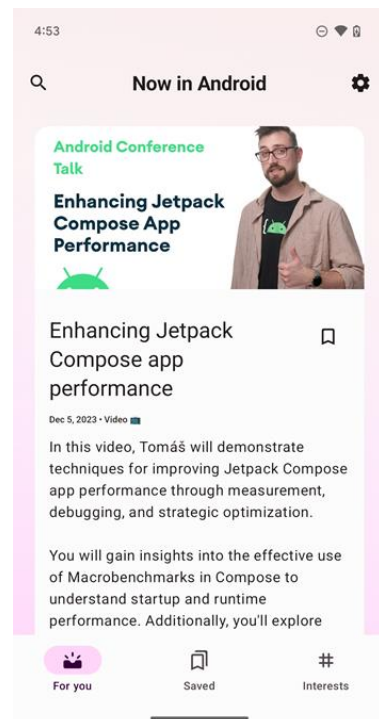
3. 典型布局场景

3.1 构建自适应导航

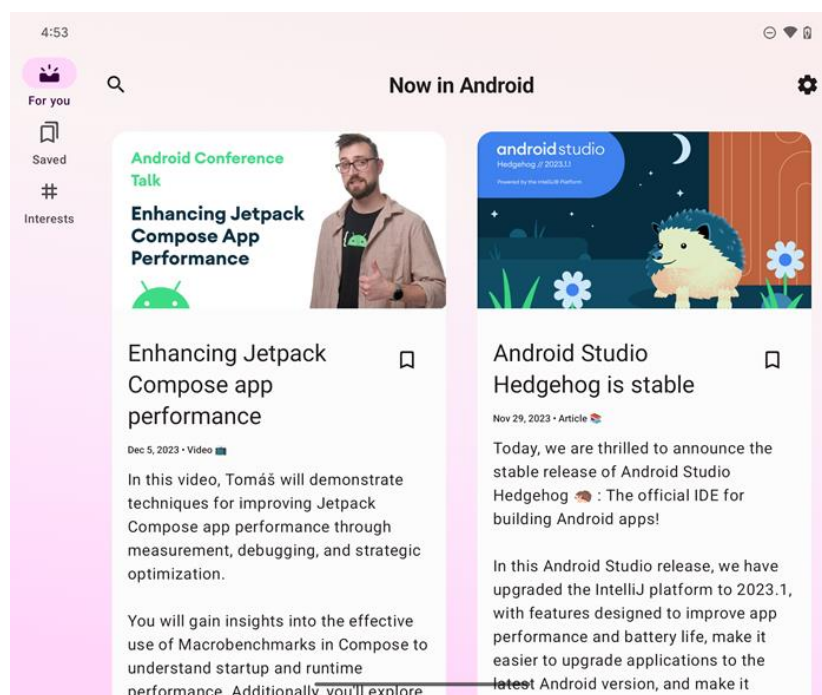
大多数应用都有几个可通过应用的主要导航界面访问的顶级目的地。在较小的窗口（例如标准手机显示屏）中，目的地通常显示在窗口底部的导航栏中。在展开的窗口（例如平板电脑上的全屏应用）中，与应用一起显示侧边导航栏通常是更好的选择，因为在握住设备的左侧和右侧时，更容易触及导航控件。

`NavigationSuiteScaffold` 会根据 `WindowSizeClass` 显示适当的导航界面可组合项，从而简化在导航界面之间切换的操作。这包括在运行时窗口大小发生变化时动态更改界面。默认行为是显示以下任一界面组件：

如果宽度或高度较小，或者设备处于桌上模式，则显示**导航栏**
侧边导航栏（适用于所有其他内容）



会在较小的窗口中显示导航栏



会在展开的窗口中显示侧边导航栏

实现方式

1. 创建框架

`NavigationSuiteScaffold` 的两个主要部分是导航套件项和所选目的地的相关内容。您可以在可组合项中直接定义导航套件项，但通常是在其他位置（例如枚举）中定义这些项：

```
enum class AppDestinations(
    @StringRes val label: Int,
    val icon: ImageVector,
    @StringRes val contentDescription: Int) {
    HOME(R.string.home, Icons.Default.Home, R.string.home),
    FAVORITES(R.string.favorites, Icons.Default.Favorite, R.string.favorites),
    SHOPPING(R.string.shopping, Icons.Default.ShoppingCart, R.string.shopping),
    PROFILE(R.string.profile, Icons.Default.AccountBox, R.string.profile),
}
```

Java

如需使用 `NavigationSuiteScaffold`，您必须跟踪当前目的地，您可以使用 `rememberSaveable` 执行此操作：

```
var currentDestination by rememberSaveable { mutableStateOf(AppDestinations.HOME) }
```

在以下示例中，`navigationSuiteItems` 参数（类型为 `NavigationSuiteScope`）使用其 `item` 函数来定义各个目的地的导航界面。目的地界面可在导航栏、侧边导航栏和抽屉式导航栏中使用。如需创建导航项，您需要循环遍历 `AppDestinations`（在上面的代码段中定义）：

```
NavigationSuiteScaffold(
    navigationSuiteItems = {
        AppDestinations.entries.forEach {
            item(
                icon = {
                    Icon(
                        it.icon,
                        contentDescription = stringResource(it.contentDescription)
                    )
                },
                label = { Text(stringResource(it.label)) },
                selected = it == currentDestination,
                onClick = { currentDestination = it }
            )
        }
    }
)
```

```

    }

    }) {

        // TODO: Destination content.
    }

```

Java

在目标内容 `lambda` 中，使用 `currentDestination` 值来确定要显示的界面。如果您在应用中使用导航库，请在此处使用该库来显示适当的目的地。当满足以下条件时，只需使用 `when` 语句即可：

```

NavigationSuiteScaffold(
    navigationSuiteItems = { /*...*/ }) {

    // Destination content.

    when (currentDestination) {

        AppDestinations.HOME -> HomeDestination()

        AppDestinations.FAVORITES -> FavoritesDestination()

        AppDestinations.SHOPPING -> ShoppingDestination()

        AppDestinations.PROFILE -> ProfileDestination()

    }
}

```

Java

2. 自定义导航栏类型

`NavigationSuiteScaffold` 的默认行为会根据窗口大小类更改导航界面。不过，您可能希望替换此行为。例如，如果您的应用为 `Feed` 显示单个大内容窗格，则该应用可以为展开的窗口使用永久性抽屉导航栏，但仍会针对紧凑和中等窗口大小类别回退到默认行为：

```

val adaptiveInfo = currentWindowAdaptiveInfo()
val customNavSuiteType = with(adaptiveInfo) {

    if (windowSizeClass.windowWidthSizeClass == WindowWidthSizeClass.EXPANDED) {

        NavigationSuiteType.NavigationDrawer

    } else {

        NavigationSuiteScaffoldDefaults.calculateFromAdaptiveInfo(adaptiveInfo)

    }
} NavigationSuiteScaffold(

    navigationSuiteItems = { /* ... */ },

    layoutType = customNavSuiteType, ) {

    // Content...
}

```

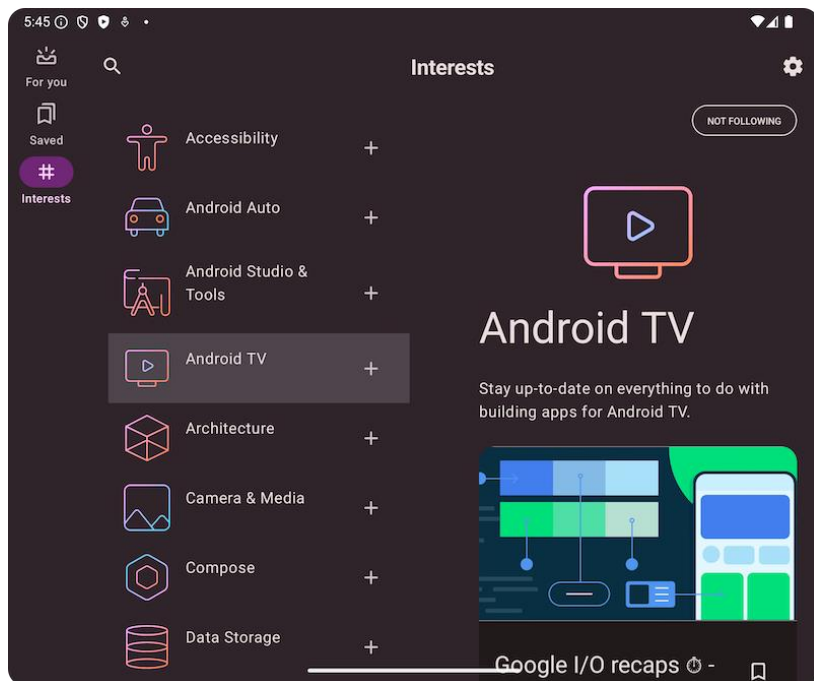
3.2 构建列表详情布局

“列表-详情”是一种界面模式，由双窗格布局组成，其中一个窗格用于显示项列表，另一个窗格用于显示从列表中选择项的详细信息。

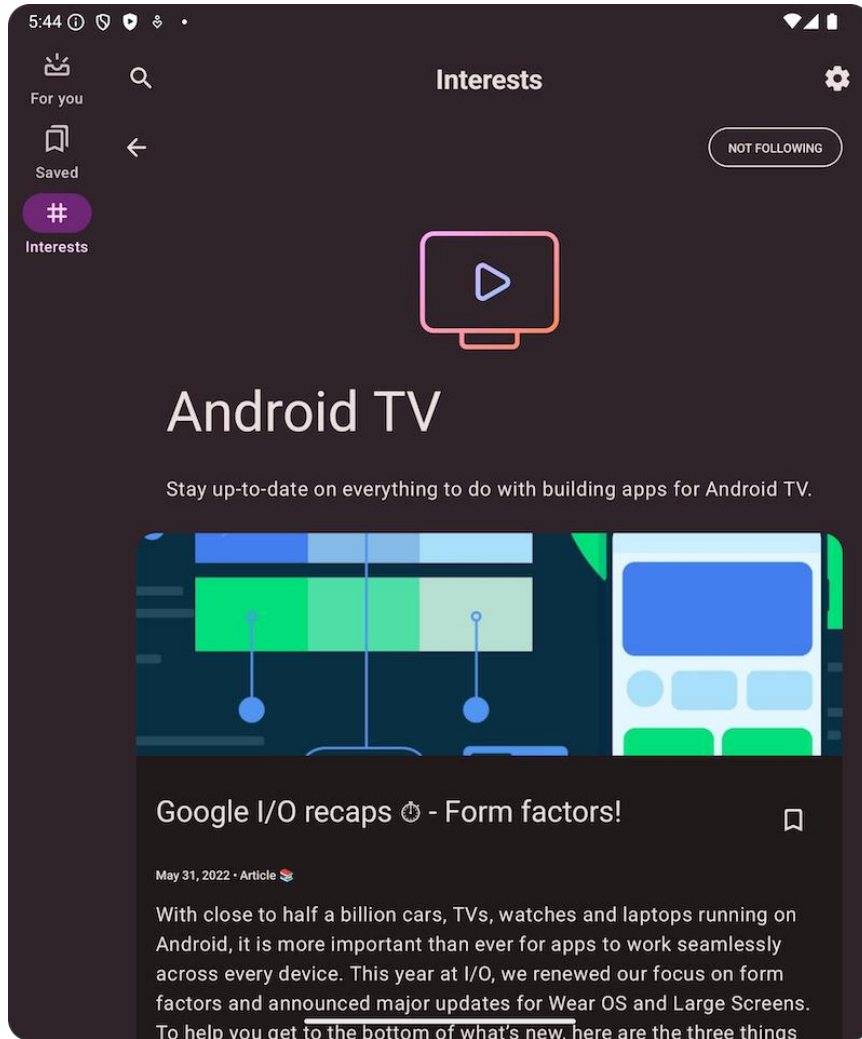
这种模式对于提供有关大型集合元素的深入信息的应用特别有用，例如，包含电子邮件列表和每封电子邮件详细内容的电子邮件客户端。列表-详情也可以用于不太重要的路径，例如将应用偏好设置划分为类别列表，并在详情窗格中显示每个类别的偏好设置。

使用 `ListDetailPaneScaffold` 实现界面模式

`ListDetailPaneScaffold` 是一种可组合项，可简化在应用中实现列表-详情模式。列表-详情架构最多可包含三个窗格：列表窗格、详情窗格和可选的额外窗格。该框架会处理屏幕空间计算。当屏幕尺寸足够大时，详情窗格会与列表窗格一起显示。在小屏幕尺寸上，框架会自动切换为全屏显示列表或详情窗格。



当屏幕尺寸足够大时，详情窗格会与列表窗格一起显示



当屏幕尺寸有限时，详情窗格（由于已选择某项内容）会占据整个空间

实现方式：

按如下方式实现 `ListDetailPaneScaffold`：

1、使用表示要选择的的内容的类。此类应为 `Parcelable`，以支持保存和恢复所选列表项。使用 `kotlin-parcelize` 插件为您生成代码。

```
@Parcelize
```

```
class MyItem(val id: Int) : Parcelable
```

2、使用 `rememberListDetailPaneScaffoldNavigator` 创建 `ThreePaneScaffoldNavigator` 并添加 `BackHandler`。此导航器用于在列表、详情和额外窗格之间移动。通过声明通用类型，导航器还会跟踪框架的状态（即正在显示哪个 `MyItem`）。由于此类型是可分块的，因此导航器可以保存和恢复状态，以自动处理配置更改。`BackHandler` 支持使用系统返回手势或按钮返回。`ListDetailPaneScaffold` 的返回按钮

的预期行为取决于窗口大小和当前的 Scaffold 值。如果 ListDetailPaneScaffold 支持使用当前状态返回，则 `canNavigateBack()` 为 `true`，从而启用 `BackHandler`。

```
val navigator =
rememberListDetailPaneScaffoldNavigator<MyItem>() BackHandler(navigator.canNavigateBack()) {
    navigator.navigateBack()}

```

Java

3、将 `navigator` 中的 `scaffoldState` 传递给 `ListDetailPaneScaffold` 可组合项。

```
ListDetailPaneScaffold(
    directive = navigator.scaffoldDirective,
    value = navigator.scaffoldValue,
    // ...)

```

Java

4、向 `ListDetailPaneScaffold` 提供列表窗格实现。使用 `AnimatedPane` 在导航期间应用默认窗格动画。然后，使用 `ThreePaneScaffoldNavigator` 导航到详情窗格 `ListDetailPaneScaffoldRole.Detail`，并显示传递的项。

```
ListDetailPaneScaffold(
    directive = navigator.scaffoldDirective,
    value = navigator.scaffoldValue,
    listPane = {
        AnimatedPane {
            MyList(
                onItemClick = { item ->
                    // Navigate to the detail pane with the passed item
                    navigator.navigateTo(ListDetailPaneScaffoldRole.Detail, item)
                }
            )
        }
    },
    // ...)

```

Java

5、在 `ListDetailPaneScaffold` 中添加详情窗格实现。导航完成后，`currentDestination` 将包含应用导航到的窗格，包括窗格中显示的内容。`content` 属性与原始 `remember` 调用中指定的类型相同（在此示例中为 `MyItem`），因此您还可以访问该属性以获取您需要显示的任何数据。

```
ListDetailPaneScaffold(  
    directive = navigator.scaffoldDirective,  
    value = navigator.scaffoldValue,  
    listPane =  
    // ...  
    detailPane = {  
        AnimatedPane {  
            navigator.currentDestination?.content?.let {  
                MyDetails(it)  
            }  
        }  
    },  
)
```

Java

整体实现代码如下：

```
val navigator =  
rememberListDetailPaneScaffoldNavigator<MyItem>() BackHandler(navigator.canNavigateB  
ack()) {  
    navigator.navigateBack()} ListDetailPaneScaffold(  
    directive = navigator.scaffoldDirective,  
    value = navigator.scaffoldValue,  
    listPane = {  
        AnimatedPane {  
            MyList(  
                onItemClick = { item ->  
                    // Navigate to the detail pane with the passed item  
                    navigator.navigateTo(ListDetailPaneScaffoldRole.Detail, item)  
                },  
            )  
        }  
    },  
)
```

```

    }

    },

    detailPane = {

        AnimatedPane {

            // Show the detail pane content if selected item is available
            navigator.currentDestination?.content?.let {

                MyDetails(it)

            }

        }

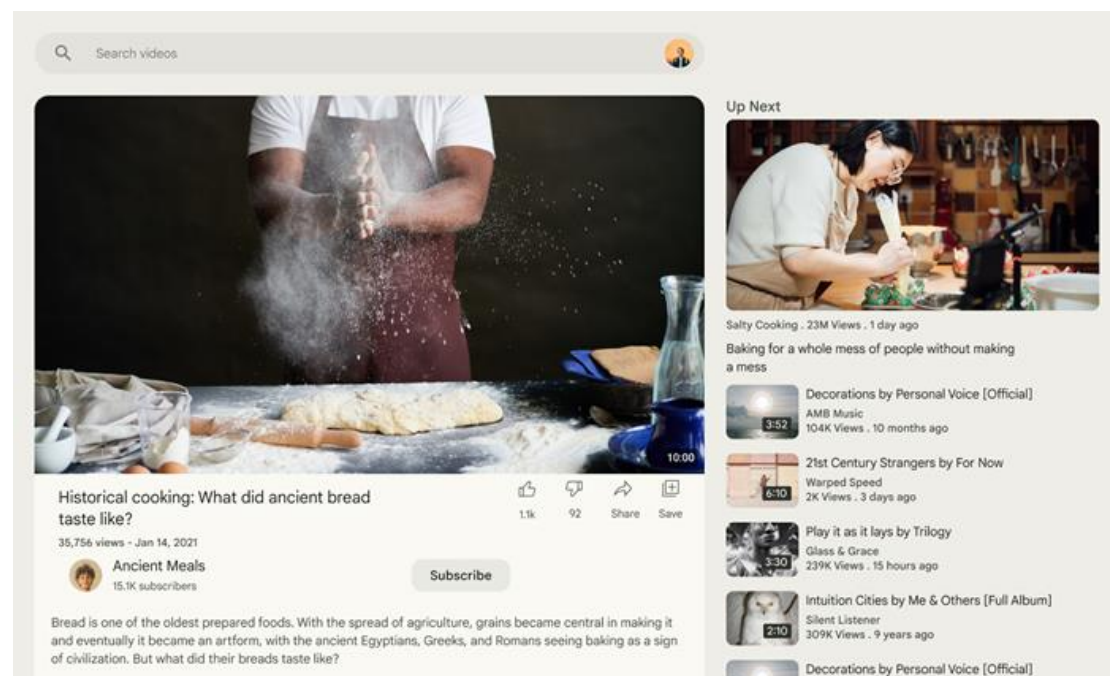
    },
),

```

Java

3.3 构建辅助窗格布局

`SupportingPaneScaffold` 最多由三个窗格组成：一个主要窗格、一个辅助窗格和一个可选的额外窗格。基架会处理将窗口空间分配给三个窗格的所有计算。在大屏幕上，该架构会显示主窗格，并在侧边显示辅助窗格。在小屏幕上，框架会全屏显示主窗格或辅助窗格。



辅助窗格布局

实现方式

1、在小窗口中，一次只能显示一个窗格，因此请使用 `ThreePaneScaffoldNavigator` 在窗格之间移动。使用 `rememberSupportingPaneScaffoldNavigator` 创建导航器的实例。如需处理返回手势，请使用可检查 `canNavigateBack()` 并调用 `navigateBack()` 的 `BackHandler`：

```
val navigator =
    rememberSupportingPaneScaffoldNavigator { BackHandler(navigator.canNavigateBack()) {
        navigator.navigateBack()
    } }
```

Java

该框架需要 `PaneScaffoldDirective`，用于控制如何拆分屏幕以及要使用的间距，还需要 `ThreePaneScaffoldValue`，用于提供窗格的当前状态（例如它们是展开还是隐藏的）。对于默认行为，请分别使用导航器的 `scaffoldDirective` 和 `scaffoldValue`：

```
SupportingPaneScaffold(
    directive = navigator.scaffoldDirective,
    value = navigator.scaffoldValue,
    mainPane = { /*...*/ },
    supportingPane = { /*...*/ },)
```

Java

主窗格和辅助窗格是包含内容的可组合项。使用 `AnimatedPane` 在导航期间应用默认窗格动画。使用 `Scaffold` 值检查辅助窗格是否处于隐藏状态；如果处于隐藏状态，则显示一个按钮，用于调用 `navigateTo(ThreePaneScaffoldRole.Secondary)` 以显示辅助窗格。

2、将 `SupportingPaneScaffold` 的各个窗格提取到各自的可组合项中，以使其可重复使用和测试。如果您想要使用默认动画，请使用 `ThreePaneScaffoldScope` 访问 `AnimatedPane`：

```
@Composable
fun ThreePaneScaffoldScope.MainPane(
    shouldShowSupportingPaneButton: Boolean,
    onNavigateToSupportingPane: () -> Unit,
    modifier: Modifier = Modifier,) {
    AnimatedPane(modifier = modifier.safeContentPadding()) {
        // Main pane content
        if (shouldShowSupportingPaneButton) {
            Button(onClick = onNavigateToSupportingPane) {
```

```

        Text("Show supporting pane")
    }
} else {
    Text("Supporting pane is shown")
}
}}@Composable

fun ThreePaneScaffoldScope.SupportingPane(
    modifier: Modifier = Modifier,) {
    AnimatedPane(modifier = modifier.safeContentPadding()) {
        // Supporting pane content
        Text("This is the supporting pane")
    }
}

```

Java

将窗格提取为可组合项可简化 SupportingPaneScaffold 的使用（将以下内容与上一部分中的架构的完整实现进行比较）：

```

val navigator =
rememberSupportingPaneScaffoldNavigator() BackHandler(navigator.canNavigateBack()) {
    navigator.navigateBack()} SupportingPaneScaffold(
    directive = navigator.scaffoldDirective,
    value = navigator.scaffoldValue,
    mainPane = {
        MainPane(
            shouldShowSupportingPaneButton = navigator.scaffoldValue.secondary ==
PaneAdaptedValue.Hidden,
            onNavigateToSupportingPane =
{ navigator.navigate(ThreePaneScaffoldRole.Secondary) }
        )
    },
    supportingPane = { SupportingPane() },)

```

Java

3.4 缩进布局

显示单列内容时，较大的屏幕会使列表与赋值内容亲密性不明确，用户难以将两侧的内容准确对应。通常情况下，需要对内容进行缩进，使用更少的栏

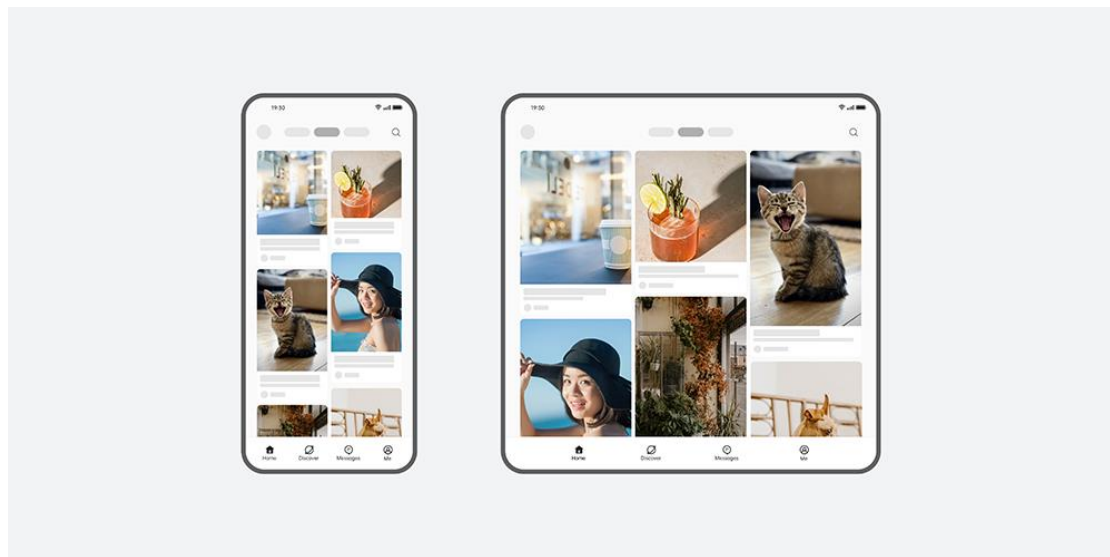


适配方法：

通过设置 Modifier 设置 `sizeIn`、`widthIn`、`heightIn` 来配置该控件的最多缩放尺寸大小，需要特别注意关于 Modifier 的编写顺序造成的影响。

3.5 增列布局：

纵向排列的卡片、宫格或瀑布流内容，在更大的屏幕上需要展示更多的列数，以控制容器和内容的尺寸，或利用增加的宽度显示更多信息



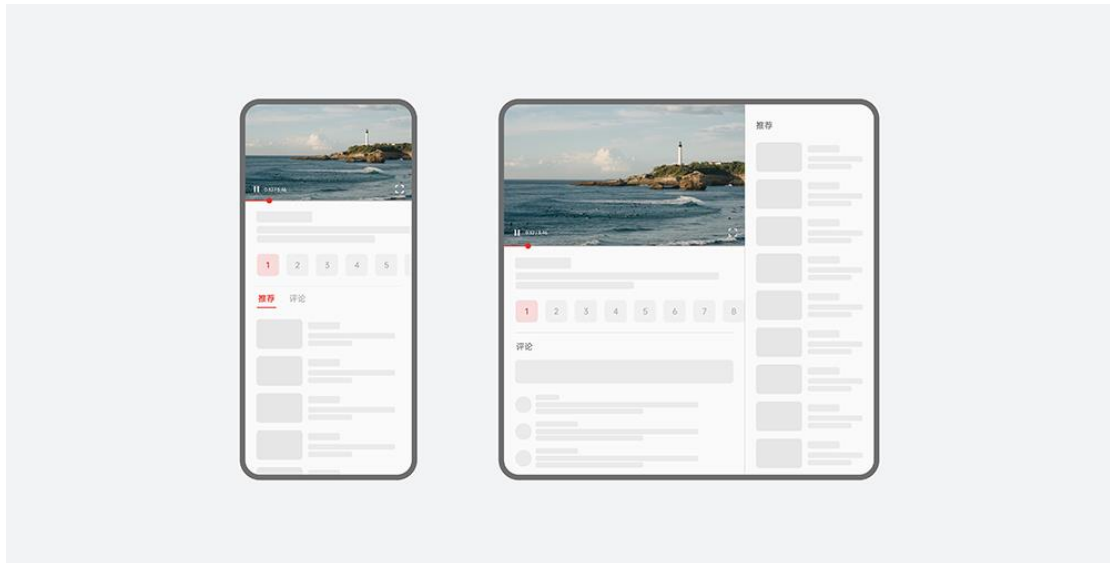
适配方法：

构建 Feed 布局

1. 在 `build.gradle.kts` 中添加依赖项目
2. 创建并使用 `LazyVerticalGrid` 动态控制列数，其中对于当前窗口的尺寸信息，可以通过 `currentWindowAdaptiveInfo().windowSizeClass` 获取，对于不同的情况展示不同的列数，大屏设备可以使用 `GridCells.Adaptive(XX.dp)` 进行动态的计算。

3.6 挪移布局

当屏幕尺寸跨越断点或比例发生变化时，部分模块的布局可以挪动位置，以提高屏幕的利用效率



适配方法：

构建辅助窗格布局

1. 在 `build.gradle.kts` 中添加依赖项目
2. 使用 `rememberSupportingPaneScaffoldNavigator` 创建导航器的实例，导航器实例中包含 `ThreePaneScaffoldValue`，用于提供窗格的当前状态（例如它们是展开还是隐藏的），通过判断可实现窗格之间移动，以及 `PaneScaffoldDirective`，用于控制如何拆分屏幕以及要使用的间距。
3. 对于处理返回手势，需要在 `BackHandler` 中去做返回处理，同样提供导航器，对导航器进行具体操作。
4. 构建 `SupportingPaneScaffold`，提供 `PaneScaffoldDirective` 与

`ThreePaneScaffoldValue`。

1. 最后填充主窗格和辅助窗格，可以使用 `AnimatedPane` 在导航期间应用默认窗格动画。同时使用 `Scaffold` 值检查辅助窗格是否处于隐藏状态；如果处于隐藏状态，则显示一个按钮，用于调用 `navigateTo(ThreePaneScaffoldRole.Secondary)` 以显示辅助窗格。

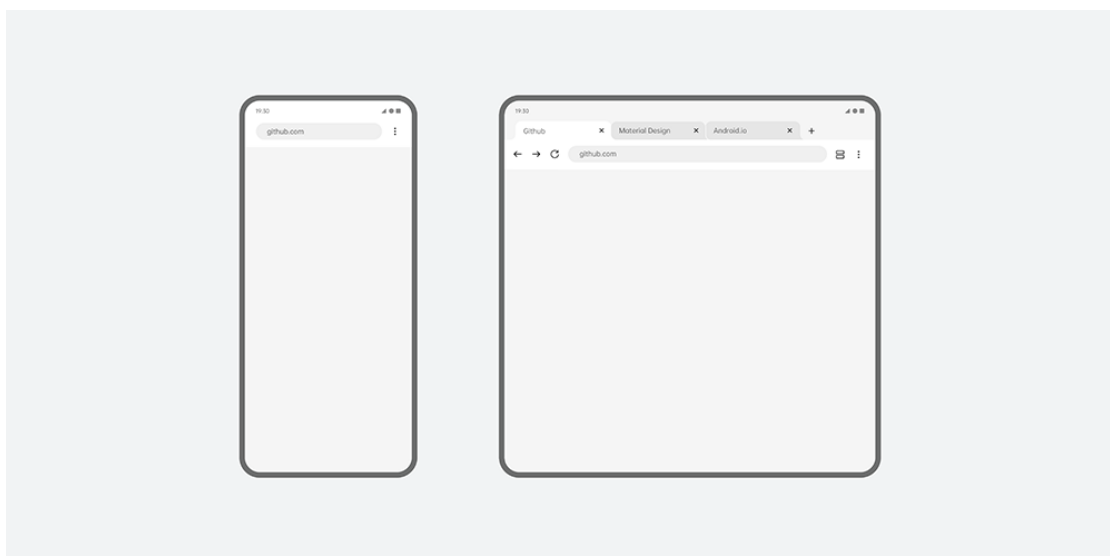
3.7 深度定制布局

跨越断点后，应用的布局完全改变。开发者可以自由发挥，进行更深度的设计与定制，展示更丰富、更多层级的信息，或者显露出更多的菜单或内容，提升用户的使用效率。

扩充层级：构建父子级与分栏布局



展开功能：减少菜单层级，在界面内直接展开更多的功能



适配方法：

构建列表详情布局

1. 在 `build.gradle.kts` 中添加依赖项目
2. 使用 `rememberListDetailPaneScaffoldNavigator` 创建导航器的实例，此导航器用于在列表、详情和额外窗格之间移动。通过声明通用类型，导航器还会跟踪框架的状态。由于此类是

可分块的，因此导航器可以保存和恢复状态，以自动处理配置更改。

1. 对于处理返回手势，需要在 `BackHandler` 中去做返回处理，同样提供导航器，对导航器进行具

体操作。

1. 构建 `ListDetailPaneScaffold`，提供 `PaneScaffoldDirective` 与

`ThreePaneScaffoldNavigator`。

- 1.

最后需要填充列表窗口的实现，并具体调用 `navigateTo` 方法提供跳转能力，以及详情窗口的具体实现。

- 2.

4. 自适应注意事项

4.1 去除宽高比的限制

由于屏幕尺寸和窗口尺寸不同，设置 `Activity` 窗口的宽高比可能会导致窗口无法充满整个屏幕，为此需要清除宽高比的设置去除宽高比的方法：通常情况下，只要我们不在清单文件中设置 `minAspectRatio` 和 `maxAspectRatio`，就不存在限制宽高比的行为

示例：

```
<?xml version="1.0" encoding="utf-8"?><manifest
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools">
    <application
        android:maxAspectRatio="2" 删除
        android:minAspectRatio="1.5"
        .....>
        .....
    </application></manifest>
```

Java

4.2 移除固定方向的限制

锁定屏幕方向并不会阻止窗口大小发生变化，反而会使窗口进入兼容模式，影响用户体验，应用需支持竖屏和横屏，不限制屏幕方向

不限制方向的方法：

1. 从 manifest 中清除 screenOrientation 设置。设置 screenOrientation 会限制 activity 的屏幕方向。

示例：

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <application>
        <activity
            android:screenOrientation="portrait"> //删除
        </activity>
    </application></manifest>
```

Java

1. 通过 Activity 的 setRequestedOrientation() 方法修改屏幕方向，使用 ViewModel 管理屏幕旋转的逻辑并与 Compose UI 交互。

示例：

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MyApp {
                ScreenOrientationViewModel(this).let { viewModel ->
                    ScreenView(viewModel)
                }
            }
        }
    }
}
```

@Composable

```
fun MyApp(content: @Composable () -> Unit) {MaterialTheme {
```

```
Box(modifier = Modifier.fillMaxSize(), contentAlignment =
    Alignment.Center) {
    content()
}
}}

@Composable
fun ScreenView(viewModel: ScreenOrientationViewModel) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = "Click the button to rotate the screen",
            fontSize = 20.sp,
            modifier = Modifier.padding(bottom = 16.dp)
        )
        Button(
            onClick = {
                viewModel.requestScreenRotation()
            },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(text = "Rotate Screen")
        }
    }
}

class ScreenOrientationViewModel(private val activity: Activity) : ViewModel() {
    fun requestScreenRotation() {
        viewModelScope.launch {
            // 获取当前活动的屏幕方向。
            val currentOrientation = activity.requestedOrientation
        }
    }
}
```

```
// 如果当前是竖屏（PORTRAIT），则新方向为横屏（LANDSCAPE）；否则，为竖屏。
val newOrientation = if (currentOrientation ==
    ActivityInfo.SCREEN_ORIENTATION_PORTRAIT) {
    ActivityInfo.SCREEN_ORIENTATION_LANDSCAPE
} else {
    ActivityInfo.SCREEN_ORIENTATION_PORTRAIT
}
activity.setRequestedOrientation(newOrientation)
}
```

Java

4.3 尺寸可调整

应用需要支持多窗口模式，支持窗口大小可调整，参与多屏幕、多窗口、多任务场景，提升用户体验和工作效率

尺寸可调整的方法：

1. 在 manifest 的 <activity> 或 <application> 元素中设置 resizableActivity 属性



示例：

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
```

```

<application
    android:name=".MyActivity"

    //true, 则 activity 能以分屏和桌面窗口模式启动;

    //false, 则 activity 不支持多窗口模式。

    android:resizeableActivity=["true" | "false"]>
</application></manifest>

```

Java

4.4 感知窗口大小，而非屏幕

要将窗口作为布局适配的基础单元，而不是屏幕，在屏幕较大的设备上，一个 activity 可能没有充满整个屏幕，或者可能会同时显示多个 activity 感知窗口的方法：

1. 在 build.gradle.kts 中添加依赖项目；
2. 获得当前窗口的大小并根据窗口大小改变布局：
 - a. 使用 `calculateWindowSizeClass()` 方法可以获得当前窗口的大小分类 `WindowSizeClass` 。
 - b. 使用 `currentWindowSize()` 方法或者可以获得当前的窗口大小，返回值为 `IntSize` 类型，其中包括 `width` 和 `height`（单位为 px），使用 `LocalDensity.current` 可获取到屏幕密度。
 - c.

`WindowMetricsCalculator.getOrCreate().computeCurrentWindowMetrics(`

`LocalContext.current).bounds` 可以获取到当前窗口的边界，再通过

`bounds.width()` 和 `bounds.height()` 方法可以获取到窗口的宽度和高度。

示例 1：

```

calculateWindowSizeClass()@OptIn(ExperimentalMaterial3WindowSizeClassApi::class)cla
ss MainActivity: ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            val windowSizeClass = calculateWindowSizeClass(this)
            WindowSizeExample(windowSizeClass)
        }
    }
}

```

```
    }}  
  
@Composable  
fun WindowSizeExample(windowSizeClass: WindowSizeClass) {  
    if (windowSizeClass.widthSizeClass == WindowWidthSizeClass.Compact) {  
        ListScreen()  
    } else {  
        TwoListScreen()  
    }  
}}
```

```
@Composable  
fun ListScreen() {  
    LazyColumn(modifier = Modifier.fillMaxSize()) {  
        // List 1  
        items(10) {  
            SimpleText("Item $it", Color.Cyan)  
        }  
        // List 2  
        items(10) {  
            SimpleText("Item $it", Color.Magenta)  
        }  
    }  
}}
```

```
@Composable  
fun TwoListScreen() {  
    Row {  
        LazyColumn(modifier = Modifier.fillMaxWidth().weight(1f)) {  
            items(10) {  
                SimpleText("Item $it", Color.Blue)  
            }  
        }  
        LazyColumn(modifier = Modifier.fillMaxWidth().weight(1f)) {  
            items(10) {  
                SimpleText("Item $it", Color.Gray)  
            }  
        }  
    }  
}
```

```

    }}
@Composable
fun SimpleText(text: String, bgColor: Color = Color.White) {
    Text(
        text = text,
        fontSize = 25.sp,
        modifier = Modifier
            .fillMaxWidth()
            .background(bgColor)
            .padding(16.dp)
    )
}

```

Java

示例 2:

```

currentWindowSize()class MainActivity: ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            val windowSize = with(LocalDensity.current) {
                currentWindowSize().toSize().toDpSize()
            }
            WindowSizeExample(windowSize)
        }
    }
}
@Composable
fun WindowSizeExample(windowSize : DpSize) {
    if (windowSize.width.value < 600) {
        ListScreen()
    } else {
        TwoListScreen()
    }
}
//其他代码与示例 1 相同

```

Java

示例 3:

解释

```
WindowMetricsCalculator.getOrCreate().computeCurrentWindowMetrics(LocalContext.current).bounds
class MainActivity: ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            var metrics =
                WindowMetricsCalculator.getOrCreate().computeCurrentWindowMetrics(LocalContext.current);
            val widthDp = metrics.bounds.width() / LocalDensity.current.density;
            val heightDp = metrics.bounds.height() / LocalDensity.current.density;
            WindowSizeExample(widthDp, heightDp)
        }
    }
}

@Composable
fun WindowSizeExample(widthDp: Float, heightDp: Float) {
    if (widthDp < 600) {
        ListScreen()
    } else {
        TwoListScreen()
    }
} //其他代码与示例 1 相同
```

Java

4.5 动态感知大小，而非静态

应用需要动态的获取感知窗口大小，而不能在进程创建时或者 Activity 创建时静态的保存使用，否则就会在屏幕或者窗口大小发生变化时，使用错误的数值进行布局，导致页面布局出现异常

动态感知窗口大小的方法：

1. 获取到当前的窗口大小后，将窗口大小作为参数传递给@Composable 函数，当窗口大小变化时，@Composable 函数会重新组合以更新布局。

示例：

解释

```
@OptIn(ExperimentalMaterial3WindowSizeClassApi::class) class MainActivity:
ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        setContent {

            val windowSizeClass = calculateWindowSizeClass(this)

            WindowSizeExample(windowSizeClass)

        }

    }

}

@Composable
fun WindowSizeExample(windowSizeClass: WindowSizeClass) {

    if (windowSizeClass.widthSizeClass == WindowWidthSizeClass.Compact) {

        ListScreen()

    } else {

        TwoListScreen()

    }

}...
```

Java